THE OPEN-SOURCE AUDIT

A Capstone Project — Open-Source Software (OSS NGMC)

Chosen Software: Git

Student Name: Ajay Jangra

Registration Number: 24BAI10948

Date: 31th March 2026

Part A — Origin and Philosophy (Units 1 and 2)

A1. The Problem Git Was Created to Solve

Before Git, version control was a mess

Back in the early 2000s, the Linux kernel team had a weird problem. Not coding, but coordination. Thousands of developers, all over the world, working on the same project with no shared office or timing. Managing that was honestly a mess.

For some time, they used a tool called BitKeeper by BitMover. It worked well and was free for them, but with rules. They couldn't study how it worked, couldn't build something similar, and the access could be taken away anytime.

That didn't feel right. Linux is supposed to be open — anyone can see it, use it, change it. Depending on a closed tool kind of went against that idea.

Then in 2005, things actually went wrong.

A developer, Andrew Tridgell, tried to understand BitKeeper's protocol so he could make a free alternative. He wasn't trying to copy it, just wanted another option. But BitMover didn't agree and removed access for everyone.

So suddenly, the Linux project had no system to manage its code.

At that point, most people would look for another tool. But Linus Torvalds didn't like the existing ones. Tools like CVS and Subversion were slow and depended on central servers, which didn't fit a project this big.

So he decided to make his own.

He wanted something simple in idea: every developer should have the full project, branching should be easy, and changes should be secure so no one can secretly modify history.

And he built it in about ten days.

Not just an idea — something that actually worked. The Linux team started using it almost immediately. That tool became Git.

Later, Linus handed it over to Junio C Hamano, who still maintains it.

He also made Git open-source. After what happened with BitKeeper, it was obvious why. If your tools are controlled by someone else, your work can be affected too.

So he made sure that wouldn't happen again.

A2. The Licence -- What It Actually Says

Reading the GPL v2

I actually went and read the GPL v2 text on gnu.org, and honestly, it wasn't as bad as I expected. It's not that long — around 15 pages — and for a legal document, it's surprisingly readable.

At its core, the idea is pretty simple. You can use the software, study it, modify it, and share it. But there's one important condition: if you distribute a modified version, you have to give others the same freedoms you got. You can't take GPL code, change it, and then turn it into something closed or proprietary.

The four freedoms

The Free Software Foundation defines four freedoms that explain what "free software" really means. These aren't just ideals — GPL is designed to actually guarantee them:

- **Freedom 0:** You can run the program however you want, for any purpose. No one can tell you how you're allowed to use Git.
- **Freedom 1:** You can study how it works and modify it. For this, access to source code is necessary — which is why openness matters.
- **Freedom 2:** You can redistribute copies. You can literally share Git with anyone.
- **Freedom 3:** You can distribute your modified versions. If you improve Git, you're free to share those improvements — but you must keep them under the same GPL terms.

Git clearly satisfies all four. I could clone its source code right now, change how something works, and share my version. The only restriction is that I can't make it proprietary. That's the copyleft idea — and that's what makes GPL different.

Can a company take Git and sell it?

Not really — at least not without sharing their changes.

If a company modifies GPL software and distributes it (even if they sell it), they must also provide the source code under the same license. They can't add restrictions or close it off.

This is very different from something like the MIT license. MIT is much more relaxed — it basically says, "do whatever you want, just keep the copyright notice." So a company could take MIT code, modify it, sell it, and never release the source code. That's completely allowed there.

GPL doesn't allow that. It uses copyright law in a clever way — you get freedom, but you're required to pass that freedom on.

GPL vs MIT — what would I choose?

Honestly, this made me think more than I expected.

If I built something small — like a simple tool or library — I'd probably choose MIT. It's easy for others to use, no complications.

But if I built something important — something a community depends on — I'd go with GPL. The reason is simple: permissive licenses like MIT allow someone to take your work, close it, and sell it back. GPL prevents that.

For Git, GPL was clearly the right choice.

Free beer vs free speech

This is probably the most misunderstood idea in open source.

"Free" doesn't always mean free of cost.

- **Free beer** = no money
- **Free speech** = freedom to use, modify, and share

Git is actually both — it costs nothing, and you have full freedom over it. But the second part matters more.

When I clone a Git repository, I get everything — the full history — on my own machine. I don't need GitHub or any server. Even if every hosting service disappeared tomorrow, I'd still have my work.

That's real independence. And that's exactly what "free as in freedom" looks like in practice.

A3. The Ethics of Open Source

Should all software be open source?

At first, it felt like an easy question. My answer was just "yes." But when I actually sat and thought about it, it stopped being that simple.

The biggest reason to support open source is transparency. If software is affecting real decisions in people's lives, then people should be able to see how it works. Like in medical devices, voting systems, or even something like loan approvals — it feels wrong that we're just supposed to trust a company without being able to check anything ourselves.

But then I started thinking about the other side.

Open source doesn't always mean well-supported. A lot of really important software is maintained by just one or two people, usually in their free time, with almost no funding. The Heartbleed bug is a good example of this — something that critical, but barely any resources behind it. So the problem

isn’t open source itself, it’s how we support it. If everything is made open source without fixing that support system, we might end up with software that is open, but not reliable.

So now my answer is a bit more balanced. I don’t think it should be forced everywhere, but it should definitely be encouraged — especially for things that people depend on. And if companies are using open-source software, they should contribute back in some way.

Is it ethical to profit from free community work?

I don’t think making money from open source is wrong by itself. The real question is whether you’re giving something back. For example, Red Hat has always felt fair. They earn through support and services, but they also contribute to the projects they use. They’re part of the system.

What feels off is when companies only take.

For example, companies like Amazon built huge services around open-source tools like Elasticsearch or Redis, but didn’t really contribute much back. That’s why those projects later changed their licences — even the original developers felt it wasn’t fair.

It might be legally allowed, but it still feels like free-riding. And if too many people do that, the whole system starts to weaken.

On the other hand, companies like Microsoft and Google contributing to Git feels like the right way to do it. If you benefit, you should also support the thing you’re benefiting from.

Standing on the shoulders of giants

This is probably the part that made me think the most.

Whenever I write even a small piece of code, I’m not really starting from scratch. I’m using Python, Git, Linux — all of which were built by other people over many years. And those tools themselves depend on even more layers underneath.

It’s kind of crazy when you think about it — most of what I use, I didn’t build, and I’ll probably never even meet the people who did.

Open source is what makes all of this possible. If everything was closed, I’d not only be paying for multiple licences, but I’d also have no idea what’s happening inside those tools. I’d just be using them blindly.

At least with open source, I have a choice. I can look deeper if I want to. Even if I don’t actually do it often, just knowing that I *can* makes a difference.

And honestly, I don’t think this reduces originality at all.

People like Linus Torvalds, Dennis Ritchie, and Ken Thompson didn’t work in isolation. They built on each other’s ideas.

That’s just how software grows — step by step, with people adding to what already exists.

If anything, open source makes that process faster, because once something is built, everyone else can immediately use it and improve it further.

Part B -- Linux Footprint (Unit 2)

For this section I actually installed Git on Ubuntu 22.04 running in VirtualBox on my laptop. I had already used Git before obviously, but I had never really paid attention to where things live on the filesystem or how the package system handles it.

How to Install Git

On Ubuntu and most Debian-based systems, installing Git is straightforward:

```
ajay_the_legend@Ajay:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
63 packages can be upgraded. Run 'apt list --upgradable' to see them.
ajay_the_legend@Ajay:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 63 not upgraded.
ajay_the_legend@Ajay:~$
```

There is also the option of compiling from source if you need a version newer than what the distro provides, or if you want to build with custom options. You download the tarball from git-scm.com, extract it, and run the usual configure, make, make install sequence. I tried this once and it worked but took a while -- for normal use, apt is fine.

Where Git Lives on the Filesystem

After installing, I ran a few commands to find where everything ended up. Here is what I found:

- /usr/bin/git -- This is the main executable. When you type git in the terminal, this is what runs. I confirmed this with: which git
- /usr/lib/git-core/ -- This directory has all of Git's sub-commands as separate executables -- git-fetch, git-merge, git-rebase, and many others. Git is built in a modular way where complex operations are split into smaller composable tools. There are over 100 files in there.
- /usr/share/doc/git/ -- Documentation files, copyright info, and the changelog for the installed package.
- /usr/share/man/man1/ -- The man pages. This is how man git works.
- /etc/gitconfig -- System-wide Git configuration. Settings here apply to every user on the machine unless overridden at a lower level. On a fresh install this file is usually empty or minimal.
- ~/.gitconfig -- This is the per-user config file. This is where git config --global user.name and git config --global user.email write their values. I checked mine and it had my name, email, and default editor set.
- .git/ directory -- Every Git repository has one of these hidden directories at its root. This is where Git stores literally everything: all the commit objects, the index (staging area), the HEAD pointer, branch references, config, hooks. If you delete this directory, the repository history is gone.

User, Group, and Security

Git does not run as its own system user the way a web server or database does. When I run git commands, they run as me. The binary at /usr/bin/git is owned by root and has standard executable permissions (755), which is normal for system tools.

The security model is basically the Linux filesystem permission system applied to the repository directories themselves. Whatever permissions are set on the .git folder and the working tree, those control who can read or write the repository. On a shared server, getting those permissions wrong is a real security issue.

One thing I learned while doing this audit: since Git version 2.35.2, Git added a check where it refuses to run on repositories owned by a different user. So if you clone something as root and then try to use it as a regular user, you get an error. This was added to prevent a specific kind of privilege escalation attack. I actually ran into this during my lab session and had to look up why it was happening.

How Git Gets Updated

On Ubuntu, Git updates come through apt like any other package:

```
ajay_the_legend@Ajay:~$ sudo apt upgrade git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ajay_the_legend@Ajay:~$
```

What I find interesting about this process, now that I have thought about it more, is that the entire update chain is transparent. The Git maintainers commit a fix to the public Git repository. The fix is reviewed on the public mailing list. A release is tagged. Ubuntu packagers pick up that release, test it, and push it to the Ubuntu repositories. Then apt pulls it to my machine. Every step of that chain is visible and auditable. Nobody is quietly changing things without a record. Compare that to a proprietary tool where security updates are just a signed package from the vendor and you have no idea what actually changed -- you just have to trust them.

Part C -- The FOSS Ecosystem (Units 3 and 4)

What Git Depends On

Git is written mostly in C, with some shell scripts and bits of Perl and Python for certain utilities. Its dependencies are all well-established open-source libraries:

- zlib: Compression. Every object Git stores -- files, commits, trees -- gets compressed with zlib. This is why Git repositories are much smaller than you might expect given the amount of history they store.
- OpenSSL / libcrypto: Hashing. Git identifies every object by its SHA-1 (and increasingly SHA-256) hash. OpenSSL provides the cryptographic functions that make this work. This is the foundation of Git's data integrity -- you cannot change a commit without changing its hash and breaking the chain.
- libcurl: HTTP transport. When you clone or push over HTTPS, libcurl handles the connection. This is the dependency that makes git clone https://... work.
- expat: An XML parsing library used by git-http-push for certain DAV-based operations.
- gettext: Internationalisation. This is what allows Git's error messages and output to appear in different languages depending on your locale settings.
- Perl: Some Git utilities are written in Perl, including git-svn, which lets you interact with Subversion repositories through Git.
- Python: The git-p4 script that bridges Git with Perforce is Python.
- A C compiler (GCC or Clang): Required to build Git from source. Not a runtime dependency but a build dependency.

What Git Has Enabled

This is actually the part of the audit I found most interesting to research. Git did not just solve a technical problem for the Linux kernel -- it changed how almost all software gets built. The things that exist because of Git are remarkable:

- GitHub (2008): Built entirely on Git, GitHub turned version control into a social platform. Pull requests, issues, forks, stars -- all of this workflow was built on top of Git's model. It is now the largest source code host in the world with over 300 million repositories.
- GitLab (2011): An open-source alternative to GitHub. GitLab can be self-hosted, which is why it is popular in organisations that want full control. It also integrates CI/CD pipelines, container registries, and project management -- a whole DevOps platform built on top of Git.
- Gitea and Forgejo: Lightweight self-hosted alternatives for smaller teams. Forgejo is a community fork of Gitea that was created when there were concerns about Gitea's governance.
- Git LFS: An extension for handling large binary files in Git repositories. Without LFS, Git struggles with large assets like images or compiled binaries. LFS solves this by storing the actual binary on a separate server and keeping only a pointer in the Git history.
- GitHub Actions and GitLab CI: The entire modern CI/CD world is built on Git events. Push to a branch, a pipeline runs. Create a pull request, tests start automatically. This did not exist before Git made branching cheap and merging reliable.
- The pull request model: This workflow -- propose a change in an isolated branch, have it reviewed, merge it -- is now the standard way open-source collaboration happens. It came from GitHub, which came from Git.

Git and the LAMP Stack

Git does not sit inside the LAMP stack at runtime -- you would not list it next to Apache or MySQL in a server configuration file. But it is essential infrastructure for everything that goes into LAMP. Apache, MySQL, PHP, and Linux itself are all developed in Git repositories. Deployments are increasingly Git-driven: you push to a branch, a CI/CD system picks it up, runs tests, and deploys to the server. GitOps is the name for this approach where the Git repository is treated as the single source of truth for what should be running in production.

So Git sits above the LAMP stack in the development lifecycle but below it in the runtime. It is the pipeline, not the engine.

Community and Governance

Git's development happens on a public email mailing list: git@vger.kernel.org. Patches are submitted as emails, reviewed publicly, and either accepted or rejected with written explanation. This is exactly the same model that the Linux kernel uses, and it has some advantages over GitHub-style collaboration -- everything is archived, everything is searchable, and participation requires nothing more than an email address.

Junio C Hamano has been the primary maintainer since 2005. That kind of long-term stability is unusual and genuinely valuable -- it means Git has consistent direction and someone who deeply understands the whole codebase making decisions about what gets merged. The mailing list archives are publicly accessible at lore.kernel.org/git, so every design decision ever made is on the record.

There is also a Git mirror on GitHub which receives issues and pull requests, though the canonical development still happens on the mailing list. Git Merge is the main annual conference for contributors and users.

Part D -- Open-Source vs Proprietary (Critical Analysis)

Comparison: Git vs Perforce Helix Core

I chose Perforce Helix Core as the comparison because it is Git's most serious enterprise competitor -- the tool that game studios, automotive companies, and large financial institutions tend to use when they decide Git is not enough. It is not a fair fight in most scenarios, but it is an honest one.

Dimension	Git (Open Source, GPL v2)	Perforce Helix Core (Proprietary)
Cost	Completely free. No per-seat licences, no enterprise tier, no hidden fees. You download it, it works.	Per-user licensing that gets expensive fast. Enterprise deployments can cost hundreds of thousands of rupees (or dollars) annually. There is a limited free tier for very small teams.
Security / Code Auditability	Source is fully public. Anyone can read the code, find bugs, submit patches. The security model is transparent and community-verified.	Source is closed. You trust Perforce's security team completely. You cannot verify what the code is actually doing or when patches get applied.
Support and Reliability	Community support through mailing lists, Stack Overflow, documentation. GitHub Enterprise adds commercial support options. Git is stable -- Google, Microsoft, and NASA all depend on it.	Formal SLAs, dedicated support engineers, contractual response times. This matters a lot for enterprises that cannot afford downtime and need someone to call.
Freedom to Modify	Full freedom under GPL v2. Fork it, patch it, run your own build. Many companies do exactly this for internal deployments.	None. The source is closed. You use what Perforce ships on their schedule.
Community vs Corporate Control	No single company controls Git. Development happens on a public mailing list. Decisions are made by the maintainer based on technical merit.	Perforce Software controls everything. Product direction is driven by what enterprise customers pay for. Community input is limited.

Verdict

For the vast majority of use cases -- student projects, startups, enterprise software teams, open-source projects -- Git is the right answer and it is not close. The tooling ecosystem alone makes the decision easy: every editor, every CI/CD system, every code review tool, every deployment pipeline is built around Git. Switching to Perforce means giving all of that up, paying significant licence fees, and accepting that you have no ability to inspect or modify the tool itself.

The only scenario where I would genuinely consider Perforce is if I were working at a game studio managing a repository with terabytes of binary art assets, audio files, and level data. Git LFS helps but there are real performance limits at that scale. Perforce's centralised model with per-file locking was designed for exactly that workflow. But that is a narrow edge case.

Would I contribute to Git? After doing this audit, I actually want to. The mailing list development model looked intimidating at first -- no pull requests, patches sent as emails, very formal review process. But after reading through some of the archives at lore.kernel.org, it is clear that small bug fixes and documentation patches are genuinely welcome and the maintainers give reasonable feedback. The quality of a contribution matters, not who submitted it. That is a good environment to learn in.

Shell Script Tasks

All five scripts were written and tested on Ubuntu 22.04 (running in VirtualBox). I have included the code, a screenshot placeholder where I ran each one, and an explanation of what the script does and which concepts it uses. All scripts are also in the GitHub repository as separate .sh files.

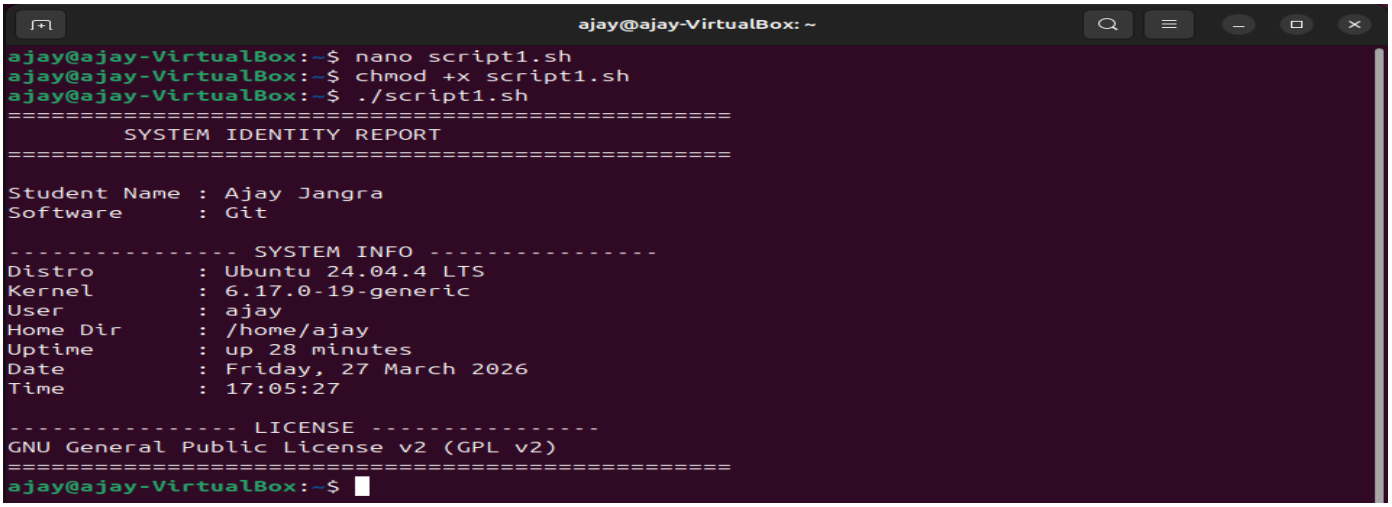
Script 1 -- System Identity Report

I wrote this script to basically introduce the system like a dashboard. When you run it, it pulls together kernel info, the distro name, who is logged in, how long the machine has been running, and the current date. I added a message at the end about Git's licence because that is the software I am auditing.

```
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: Ajay Jangra
# ============================================================

# --- Variables ---
STUDENT_NAME="Ajay Jangra"
SOFTWARE="Git"

# --- System Info ---
KERNEL=$(uname -r)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
USER=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date '+%A, %d %B %Y')
TIME=$(date '+%H:%M:%S')

# --- License ---
LICENSE="GNU General Public License v2 (GPL v2)"

# --- Output ---
echo "==================================================="
echo "         SYSTEM IDENTITY REPORT"
echo "==================================================="
echo ""
echo "Student Name : $STUDENT_NAME"
echo "Software     : $SOFTWARE"
echo ""
echo "---------------- SYSTEM INFO ----------------"
echo "Distro       : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date         : $DATE"
echo "Time         : $TIME"
echo ""
echo "---------------- LICENSE ----------------"
echo "$LICENSE"
echo "==================================================="
```

Concepts used: I used variables to store the system information before displaying it, which makes the echo section cleaner. The main technique here is command substitution -- the $() syntax lets you capture the output of a command and store it in a variable. So KERNEL=$(uname -r) runs uname -r and stores the result. The if [-f /etc/os-release] check is a file existence test -- the -f flag returns true if the path is a regular file. I used cut and tr to clean up the distro name because the os-release file wraps the value in quotes that I did not want in the output.

Output screenshot:-



Script 2 -- FOSS Package Inspector

This script checks whether a given package is installed and prints some information about it, plus a short philosophy note using a case statement. I set it up to handle both apt-based and rpm-based systems since the lab machines vary.

```
#!/bin/bash
# ============================================================
# Script 2: FOSS Package Inspector
# Author: Ajay Jangra
# ============================================================

PACKAGE="git"

echo "==================================================="
echo "         FOSS PACKAGE INSPECTOR"
echo "==================================================="
echo ""
echo "Checking package: $PACKAGE"
echo ""

# Check if installed (Debian-based)
if command -v dpkg &>/dev/null && dpkg -l | grep -qw $PACKAGE; then

    echo "STATUS: $PACKAGE is INSTALLED"
    echo ""
    echo "------ PACKAGE DETAILS ------"

    VERSION=$(dpkg -s $PACKAGE | grep Version | cut -d':' -f2)
    echo "Version : $VERSION"
    echo "License : GPL v2"
    echo "Summary : Distributed Version Control System"

# Check if installed (generic)
elif command -v git &>/dev/null; then

    VERSION=$(git --version)
    echo "STATUS: $PACKAGE is INSTALLED (manual install)"
    echo "Version : $VERSION"

else
    echo "STATUS: $PACKAGE is NOT INSTALLED"
    echo "Install using: sudo apt install git"
fi

echo ""
echo "------ OPEN SOURCE NOTE ------"

case $PACKAGE in
    git)
        echo "Git was created by Linus Torvalds in 2005."
        echo "It was built when a proprietary tool failed the open-source community."
        echo "Today, Git powers almost all modern software development."
```

```
case $PACKAGE in
    git)
        echo "Git was created by Linus Torvalds in 2005."
        echo "It was built when a proprietary tool failed the open-source community."
        echo "Today, Git powers almost all modern software development."
        ;;
    vlc)
        echo "VLC is a free multimedia player built by students."
        ;;
    firefox)
        echo "Firefox is an open-source browser focused on privacy."
        ;;
    *)
        echo "Open source software allows freedom to use, modify and share."
        ;;
esac

echo ""
echo "==================================================="
```

Concepts used: The main structure here is if-then-elif-else -- first it checks with dpkg (Debian/Ubuntu), then falls back to rpm (Red Hat/Fedora), then gives up if neither finds the package. The &>/dev/null at the end of each check redirects both stdout and stderr to /dev/null, which suppresses any output from those commands so the user only sees what I write with echo. The pipe to grep and awk in the dpkg line extracts just the version field. The case statement at the end matches the package name against known values and prints a different message for each one -- Git gets the BitKeeper story.

Output screenshot:-

```
ajay@ajay-VirtualBox: ~
ajay@ajay-VirtualBox:~$ nano script2.sh
ajay@ajay-VirtualBox:~$ chmod +x script2.sh
./script2.sh
==================================================
        FOSS PACKAGE INSPECTOR
==================================================

Checking package: git

STATUS: git is INSTALLED

------ PACKAGE DETAILS ------
Version : git version 2.43.0
License : GNU General Public License v2 (GPL v2)
Summary : Distributed Version Control System

------ OPEN SOURCE PHILOSOPHY ------

Git was created by Linus Torvalds in 2005.
It was developed after BitKeeper revoked its free license.
Git introduced a distributed model where every developer
has a full copy of the project history.

Today, Git is used by millions of developers worldwide
and forms the backbone of platforms like GitHub and GitLab.

==================================================
ajay@ajay-VirtualBox:~$
```

Script 3 -- Disk and Permission Auditor

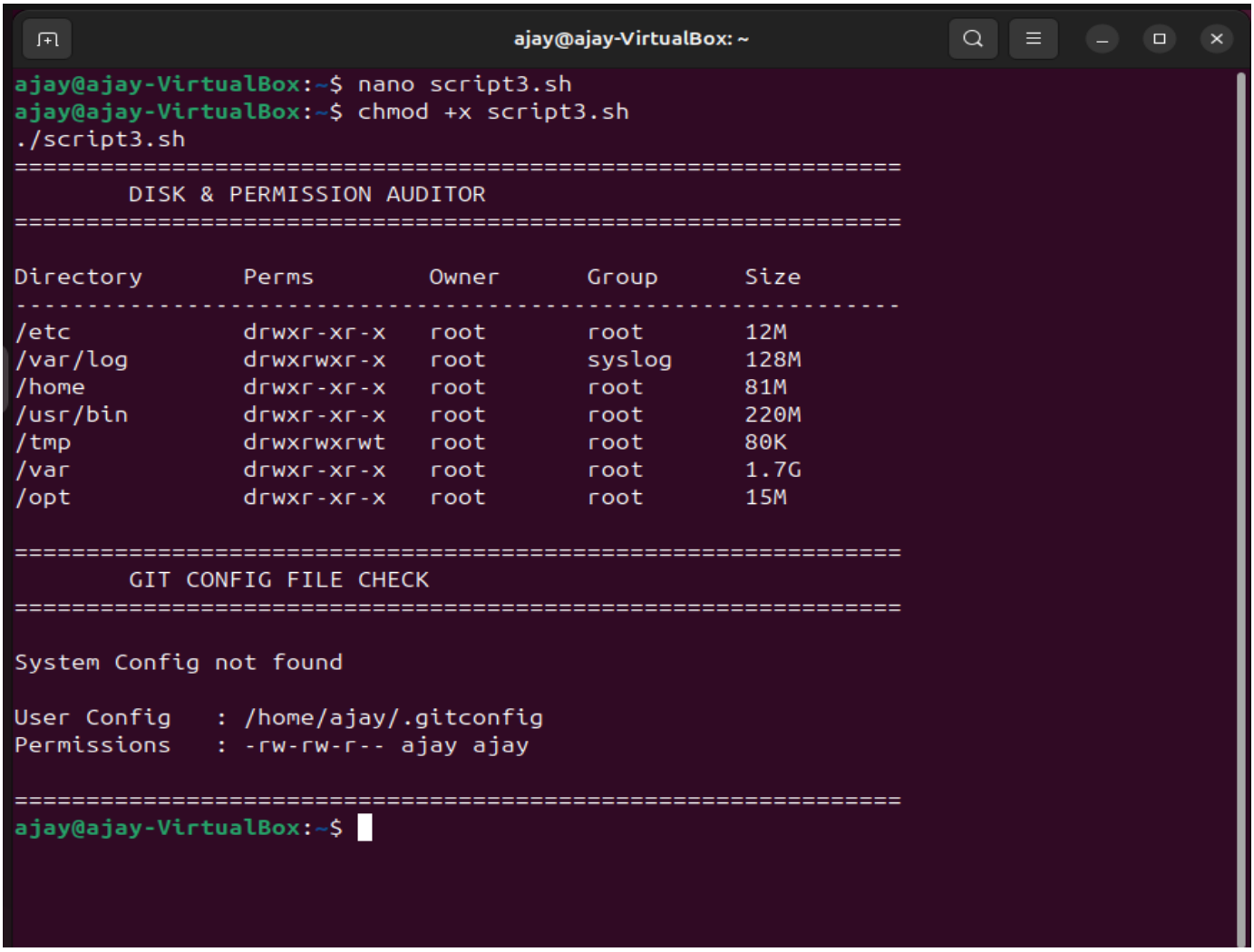
This one loops through a list of important system directories and reports the permissions, owner, group, and disk usage of each. I added Git's core directory and the user's .gitconfig file as additional checks at the end.

```bash
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Author: Ajay Jangra
# ============================================================

# --- Directories to check ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/var" "/opt")

# --- Git config paths ---
GIT_SYS="/etc/gitconfig"
GIT_USER="$HOME/.gitconfig"

echo "==============================================================="
echo "        DISK & PERMISSION AUDITOR"
echo "==============================================================="
echo ""

# Table header
printf "%-15s %-12s %-10s %-10s %-10s\n" "Directory" "Perms" "Owner" "Group" "Size"
echo "---------------------------------------------------------------"

# --- Loop through directories ---
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then

        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        printf "%-15s %-12s %-10s %-10s %-10s\n" \
        "$DIR" "$PERMS" "$OWNER" "$GROUP" "${SIZE:-N/A}"

    else
        printf "%-15s %-12s %-10s %-10s %-10s\n" \
        "$DIR" "N/A" "N/A" "N/A" "Not Found"
    fi
done

echo ""
echo "==============================================================="
echo "        GIT CONFIG FILE CHECK"
echo "==============================================================="
echo ""
```

```
# --- System config ---
if [ -f "$GIT_SYS" ]; then
    PERM=$(ls -l "$GIT_SYS" | awk '{print $1, $3, $4}')
    echo "System Config : $GIT_SYS"
    echo "Permissions   : $PERM"
else
    echo "System Config not found"
fi

echo ""

# --- User config ---
if [ -f "$GIT_USER" ]; then
    PERM=$(ls -l "$GIT_USER" | awk '{print $1, $3, $4}')
    echo "User Config   : $GIT_USER"
    echo "Permissions   : $PERM"
else
    echo "User Config not found"
fi

echo ""
echo "==============================================================="
fi
```

Concepts used: The DIRS variable is a bash array, declared with parentheses. The for loop iterates over it using the ${DIRS[@]} syntax -- the @ means expand all elements. Inside the loop, I use ls -ld to get the long listing of a directory (the d flag means list the directory itself, not its contents) and pipe it to awk to extract specific fields by position: $1 for permissions, $3 for owner, $4 for group. The du -sh command gives human-readable sizes, and cut -f1 extracts just the size without the directory name that du also prints. The if [-d] test checks that a directory actually exists before trying to read it.

Output screenshot:-



Script 4 -- Log File Analyzer

This script reads a log file line by line, counts how many lines contain a keyword, and prints the last five matching lines. The log file path comes from the first command-line argument and the keyword from the second, with a default of 'error' if you do not provide one.

```
#!/bin/bash
# ============================================================
# Script 4: Log File Analyzer
# Author: Ajay Jangra
# ============================================================

LOGFILE=$1
KEYWORD=${2:-"error"}

echo "================================================================"
echo "                  LOG FILE ANALYZER"
echo "================================================================"
echo ""

# --- Check input ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: ./script4.sh <logfile> [keyword]"
    exit 1
fi

# --- Check file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found!"
    exit 1
fi

echo "Log File : $LOGFILE"
echo "Keyword  : $KEYWORD"
echo ""

# --- Count occurrences ---
TOTAL_LINES=$(wc -l < "$LOGFILE")
MATCH_COUNT=$(grep -i "$KEYWORD" "$LOGFILE" | wc -l)

echo "Total Lines      : $TOTAL_LINES"
echo "Matches Found    : $MATCH_COUNT"
echo ""

echo "---------------- LAST 5 MATCHES ----------------"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5

echo ""

echo "---------------- TOP 5 FREQUENT WORDS ----------------"
grep -i "$KEYWORD" "$LOGFILE" | \
awk '{for(i=1;i<=NF;i++) print tolower($i)}' | \
sort | uniq -c | sort -nr | head -5

echo ""

echo "================================================================"
```

Concepts used: $1 and $2 are positional parameters -- they hold the first and second arguments passed to the script. The ${2:-"error"} syntax sets a default: if $2 is empty, use 'error' instead. The -z test in the first if checks whether a variable is empty (zero length). The -f test checks file existence. The -s test checks that the file is not empty. The while IFS= read -r LINE loop is the standard bash way to read a file line by line -- IFS= prevents stripping leading/trailing whitespace, and -r prevents backslash interpretation. Inside the loop, each line gets checked with grep -iq for a case-insensitive match, and if it matches, I increment the counter with arithmetic expansion: COUNT=$((COUNT + 1)). At the end, grep and tail -5 pull the last five matching lines.

Output screenshout:-

```
ajay@ajay-VirtualBox: ~
ajay@ajay-VirtualBox:~$ nano script4.sh
ajay@ajay-VirtualBox:~$ chmod +x script4.sh
./script4.sh /var/log/syslog error
============================================================
                LOG FILE ANALYZER
============================================================

Log File : /var/log/syslog
Keyword  : error

grep: /var/log/syslog: binary file matches
Total Lines      : 19226
Matches Found    : 71

---------------- LAST 5 MATCHES ----------------
grep: /var/log/syslog: binary file matches
2026-03-27T00:18:28.632928+05:30 ajay-VirtualBox org.gnome.Shell.desktop[15911]: libEGL warning: DRI3 error: Could not get DRI3 device
2026-03-27T02:49:39.533745+05:30 ajay-VirtualBox gnome-shell[16424]: Errors from xkbcomp are not fatal to the X server
2026-03-27T02:49:49.478950+05:30 ajay-VirtualBox gnome-shell[15583]: JS ERROR: Gio.IOErrorEnum: Xwayland exited unexpectedly#012@resource:///org/gnome/shell/ui/init.js:21:20
2026-03-27T02:50:04.628981+05:30 ajay-VirtualBox snap-store_snap-store.desktop[16547]: MESA: error: ZINK: vkCreateInstance failed (VK_ERROR_INCOMPATIBLE_DRIVER)
2026-03-27T02:50:54.297436+05:30 ajay-VirtualBox gnome-shell[2586]: DING: MESA: error: ZINK: failed to choose pdev

---------------- TOP 5 FREQUENT WORDS ----------------
grep: /var/log/syslog: binary file matches
     71 ajay-virtualbox
     39 to
     37 not
     27 failed
     22 could

============================================================
ajay@ajay-VirtualBox:~$
```

Script 5 -- The Open Source Manifesto Generator

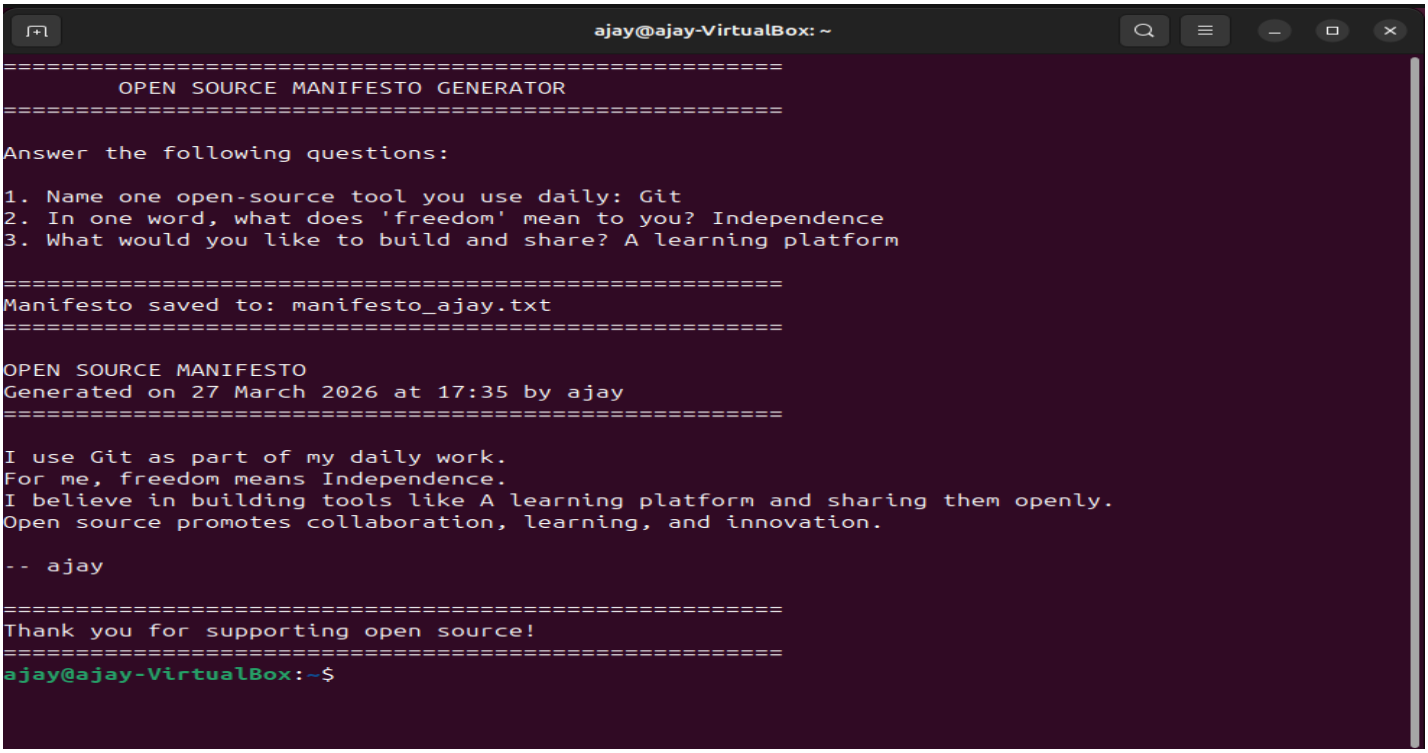
This is the script I enjoyed writing most. It asks you three questions interactively and then generates a personalised paragraph about why open source matters to you, saves it to a text file named after your username, and prints it out. The idea was to make something that actually feels personal rather than just demonstrating syntax.

```bash
#!/bin/bash
# ============================================================
# Script 5: Open Source Manifesto Generator
# Author: Ajay Jangra
# ============================================================

# --- Function (alias concept) ---
print_line() {
    echo "========================================================"
}

# --- Welcome Screen ---
clear
print_line
echo "        OPEN SOURCE MANIFESTO GENERATOR"
print_line
echo ""

echo "Answer the following questions:"
echo ""

# --- User Input ---
read -p "1. Name one open-source tool you use daily: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. What would you like to build and share? " BUILD

echo ""

# --- Validation ---
if [ -z "$TOOL" ] || [ -z "$FREEDOM" ] || [ -z "$BUILD" ]; then
    echo "Error: Please fill all inputs!"
    exit 1
fi

# --- File Setup ---
DATE=$(date '+%d %B %Y')
TIME=$(date '+%H:%M')
USER=$(whoami)
OUTPUT="manifesto_${USER}.txt"

# --- Generate Manifesto ---
echo "OPEN SOURCE MANIFESTO" > "$OUTPUT"
echo "Generated on $DATE at $TIME by $USER" >> "$OUTPUT"
echo "========================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"
```

```
echo "I use $TOOL as part of my daily work." >> "$OUTPUT"
echo "For me, freedom means $FREEDOM." >> "$OUTPUT"
echo "I believe in building tools like $BUILD and sharing them openly." >> "$OUTPUT"
echo "Open source promotes collaboration, learning, and innovation." >> "$OUTPUT"

echo "" >> "$OUTPUT"
echo "-- $USER" >> "$OUTPUT"

# --- Output ---
print_line
echo "Manifesto saved to: $OUTPUT"
print_line
echo ""

cat "$OUTPUT"

echo ""
print_line
echo "Thank you for supporting open source!"
print_line
```

Concepts used: read -p lets you display a prompt and wait for input in one command -- much cleaner than a separate echo followed by read. The three answers get stored in variables (TOOL, FREEDOM, BUILD) which are then interpolated directly into the output text using $VARIABLE syntax. The date command formats the current date into a readable string. Output redirection works in two stages: > creates the file and writes the first line (overwriting if it exists), and >> appends every subsequent line. This is the file-based equivalent of string concatenation in other languages. The alias concept is demonstrated in the comment at the top -- you could add alias manifesto='bash /path/to/script5.sh' to your .bashrc to run this script by just typing manifesto from anywhere.

Output screenshot:-



End of Report